

Linux Server Operations Lab

Phase 6 Review

PythonによるAnsible実行結果の解析・状態判定

1. Phase 6 の目的

Phase 6では、Ansibleが取得したLinuxサーバの状態をPythonで解析し、単なる生のコマンド出力ではなく「OK / WARNING / FAILED / UNREACHABLE」という運用しやすい状態へ変換しました。

ここが、Phase 5の「情報収集」とPhase 7の「Excelへの自動書き戻し」をつなぐ中間層です。

2. 最初の確認 - PythonからAnsibleを実行

```
result = subprocess.run(
    ['ansible', 'servers', '-i', inventory, '-a', 'df -P /'],
    capture_output=True,
    text=True
)
```

```
print(result.stdout)
print(result.stderr)
print(result.returncode)
```

`subprocess.run()` でAnsibleを起動し、`stdout` / `stderr` / `returncode` を取得しました。`returncode=0` ならコマンド実行自体は正常です。

3. Disk使用率の抽出

Ansibleのdf出力から、Pythonの正規表現でルートファイルシステムの使用率だけを抜き出しました。

```
disk_match = re.search(r'(\d+)\s+\s+\s+', line)
```

```
if disk_match:
    disk_percent = int(disk_match.group(1))
```

Hostname	Disk使用率
tokyo	25%
yokohama	16%
omiya	16%

判定ルール

```
if disk_percent >= 80:
    status = 'WARNING'
else:
    status = 'OK'
```

今回は学習用ルールとして80%以上をWARNING、それ未満をOKとしました。実務では環境に合わせて閾値を決めます。

4. nginx状態の解析

次に `systemctl is-active nginx`

を Ansible から実行し、active なら OK、それ以外の失敗状態を FAILED として扱いました。

```
if line.strip() == 'active' and current_host:
    nginx_status[current_host] = 'OK'
```

5. 最終ステータスの統合

Disk と nginx を別々に確認したあと、サーバ単位で最終判定を作成しました。異常度の高い状態を優先する設計です。

```
if disk['status'] == 'UNREACHABLE' or nginx == 'UNREACHABLE':
    final = 'UNREACHABLE'
elif disk['status'] == 'FAILED' or nginx == 'FAILED':
    final = 'FAILED'
elif disk['status'] == 'WARNING':
    final = 'WARNING'
else:
    final = 'OK'
```

状態	意味
OK	点検項目に問題なし
WARNING	接続はできるが閾値超過など注意が必要
FAILED	接続はできるがコマンド・サービス確認に失敗
UNREACHABLE	Ansibleが対象サーバへ接続できない

6. UNREACHABLE の意図的テスト

異常判定が本当に動くか確認するため、テスト用 inventory をコピーし、omiya の IP を 192.168.56.99 に変更して到達不能な状態を作りました。

```
cp hosts_generated.ini hosts_test_unreachable.ini
sed -i 's/192.168.56.13/192.168.56.99/' hosts_test_unreachable.ini
```

```
omiya Disk=- nginx=UNREACHABLE Result=UNREACHABLE
tokyo Disk=25% nginx=OK Result=OK
yokohama Disk=16% nginx=OK Result=OK
```

単に「正常系が動いた」だけでなく、接続不能を正しく UNREACHABLE に分類できることまで確認しました。

7. FAILED の意図的テスト

次に、存在しない `nginx_not_exist` を確認するよう一時変更し、接続できるがサービス確認に失敗するケースを作りました。3台すべてをFAILEDとして判定できました。

```
systemctl is-active nginx_not_exist
```

8. 発生したトラブル - U+3000

Pythonファイル編集時に全角スペースが混入し、`SyntaxError: invalid non-printable character U+3000` が発生しました。見た目では分かりにくいいため、最終的にはファイル全体をヒアドキュメントで上書きして復旧しました。

このトラブルから、コードでは半角スペースを使うこと、コピー&ペースト後に見えない文字が混入する可能性があることを学びました。

9. 正常状態への復旧

テスト後はinventoryとnginx確認コマンドを正常状態へ戻し、最終的に3台すべてOKとなることを再確認しました。

```
omiya Disk=16% nginx=OK Result=OK
tokyo Disk=25% nginx=OK Result=OK
yokohama Disk=16% nginx=OK Result=OK
```

10. このPhaseで理解したこと

Ansibleの出力は、そのままでは人間向けのログです。Pythonで必要な値だけを抽出し、状態をルール化することで、後続のExcel・通知・ダッシュボードなどが扱いやすい構造化データへ変換できます。

また、正常系だけでなく UNREACHABLE / FAILED

を意図的に発生させて確認したことで、異常系を含めた運用処理として一段階実践的になりました。

11. Phase 7へのつながり

Phase 6で作った判定結果を、Phase 7では `openpyxl` を使ってExcelの `02_CHECK_RESULTS` シートへ書き戻します。これにより「Excel → Python → Ansible → Linux → Python → Excel」の循環が完成します。

12. note記事候補

「Ansibleの実行結果をPythonで解析する」「UNREACHABLEとFAILEDをどう分けるか」「正規表現でdf出力からDisk使用率を取得する」「正常系だけでなく異常系まで試すインフラ自動化学習」といった記事へ展開できます。